

Full Stack Development

1. What are Template Literals?

👉 Used to create strings using **backticks (`)**

✓ Allows:

- Multi-line strings
- Variables inside string using `${}`

📌 Example:

```
let name = "Manish";  
console.log(` Hello ${name} `);
```

🧠 **Memory Trick:**

👉 “Backtick + `${}` = Template String”

2. Purpose of Spread Syntax (...)

👉 Used to **expand elements**

✓ Uses:

- Copy arrays
- Merge arrays
- Pass multiple values

📌 Example:

```
let arr1 = [1,2];  
let arr2 = [...arr1,3,4];
```

🧠 Trick:

👉 “... = spread everything”

3. Arrow Functions (ES6)

👉 Short way to write functions

📌 Example:

```
const add = (a,b) => a + b;
```

✓ No function keyword

✓ Short syntax

4. Symbols in JavaScript

👉 Unique and immutable value

✓ Used for:

- Unique object keys

📌 Example:

```
let id = Symbol("id");
```

💡 Trick:

👉 “Symbol = Unique Identity”

5. Advantages of TypeScript

✓ Type safety

✓ Better error detection

✓ Better IDE support (auto-suggestion)

💡 Trick:

👉 “Safe + Error + Smart”

6. Decorators in TypeScript

👉 Special functions to modify classes/methods

📌 Example:

```
@log  
class Test {}
```

✓ Used for:

- Logging
 - Validation
 - Authorization
-

7. What are let and const?

👉 New variable types in ES6

✓ let → can change value

✓ const → cannot change value

🧠 Trick:

👉 “let = change, const = fixed”

8. Arrow vs Regular Function

Arrow Function	Regular Function
----------------	------------------

Short	Long
-------	------

No this binding	Has this
-----------------	----------

Simple syntax	Complex
---------------	---------

9. Destructuring Assignment

👉 Extract values easily

📌 Array:

```
let [a,b] = [1,2];
```

📌 Object:

```
let {name} = {name:"Manish"};
```

🧠 Trick:

👉 “Break structure → Destructuring”

10. Iterators & Generators

👉 Used to control loops

✓ Iterator → manual iteration

✓ Generator → function with yield

📌 Example:

```
function* gen(){  
  yield 1;  
  yield 2;  
}
```

🧠 Trick:

👉 “Generator = pause & continue”

11. Pure Functions

👉 Same input → same output

✓ No side effects

📌 Example:

```
function add(a,b){  
  return a+b;  
}
```

💡 Trick:

👉 “No change outside → Pure”

12. Async & Await

👉 Makes async code easy

✓ Avoids callbacks

✓ Looks like normal code

📌 Example:

```
async function test(){  
  let data = await fetch(url);  
}
```

💡 Trick:

👉 “Async = wait like sync”

13. OOP in ES6

👉 Programming using objects & classes

✓ Concepts:

- Class
- Object
- Inheritance
- Encapsulation

📌 Example:

```
class Car {  
  constructor(name){  
    this.name = name;  
  }  
}
```

14. Decorators Use Cases

- ✓ Logging
 - ✓ Security
 - ✓ Validation
 - ✓ Performance tracking
-

15. Type Narrowing (TypeScript)

👉 Reducing variable type

📌 Example:

```
if(typeof x === "string"){  
  // x is string here  
}
```

🧠 Trick:

👉 “Check type → Narrow type”

16. Type Guards

👉 Used to check type

✓ typeof

✓ instanceof

📌 Example:

```
if(typeof x === "number")
```

17. What is MEAN Stack?

👉 Full-stack JavaScript framework

✓ M → MongoDB

✓ E → Express

✓ A → Angular

✓ N → Node.js

💡 Trick:

👉 “M-E-A-N = Full Stack JS”

18. MEAN Stack Architecture

👉 Flow:

Client (Angular)



Server (Node + Express)



Database (MongoDB)

19. Technologies in MEAN Stack

✓ MongoDB → Database

✓ Express → Backend framework

✓ Angular → Frontend

✓ Node.js → Server runtime